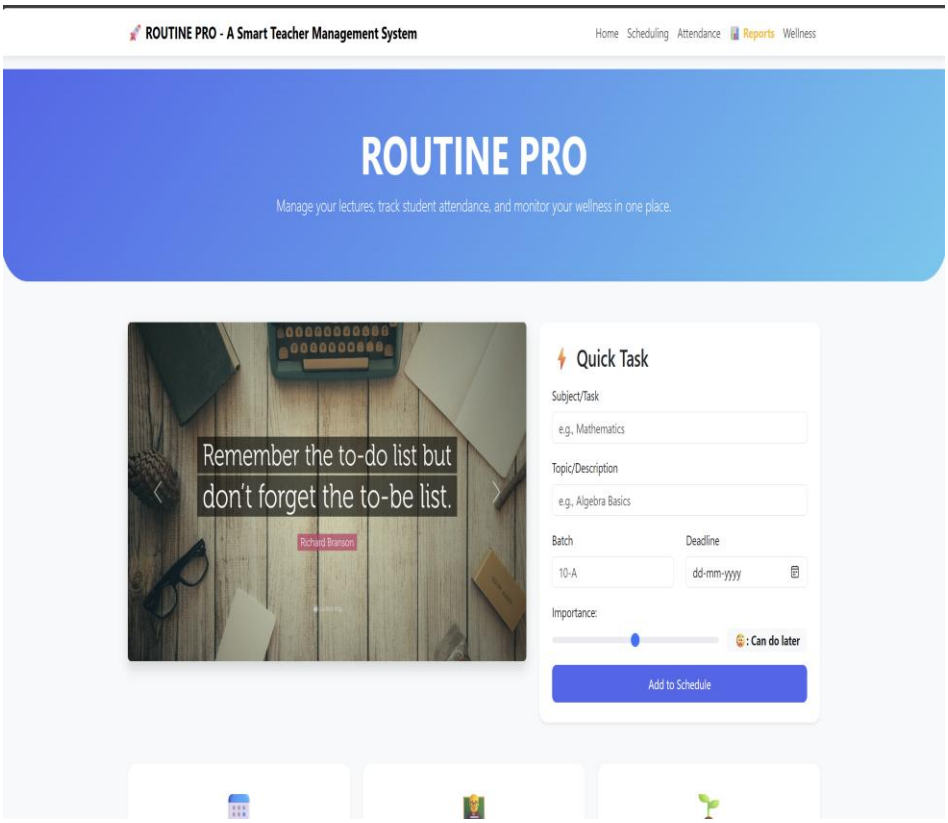

Routine Pro – A Smart Teacher Management System

PROJECT REPORT

Routine Pro is a web-based application designed to help teachers manage lecture schedules, track student attendance, and maintain daily wellness logs in one unified platform.

Saanvi Awasthi
Reg No – 25BCE10478
B.Tech (CSE – Core)

Introduction

In today's education environment, teachers are responsible for managing lecture schedules, maintaining attendance records, and handling personal workload efficiently. Traditional methods often involve fragmented tools or manual processes, leading to inefficiency and lack of insights.

Routine Pro is an AI-enhanced web application that integrates scheduling, attendance tracking, analytics, and wellness monitoring into a single unified system.

Problem Statement

Teachers face multiple challenges:

- Difficulty in managing lecture schedules efficiently
- Manual and error-prone attendance tracking
- Lack of insights into student performance
- No system to monitor workload and stress levels
- Fragmented tools instead of a unified platform

Existing solutions are either fragmented or overly complex. There is a need for a unified, user-friendly system.

Objectives

The main objectives of this project are:

- Build a centralized teacher management system
- Automate attendance tracking and reporting
- Provide AI-based performance insights
- Monitor teacher workload and stress levels
- Ensure a clean, simple, and user-friendly interface

Methodology

The project follows a modular approach:

- **Frontend:** Developed using HTML, CSS, and Bootstrap for responsive design
- **Backend:** Implemented using Python Flask framework

- **Database:** SQLite database managed using SQLAlchemy ORM
- **Machine Learning:** Scikit-learn (Decision Tree)

Steps:

1. Requirement analysis of teacher needs
2. Designing database schema
3. Implementing backend logic using Flask
4. Creating user interface with Bootstrap
5. Integrating modules and testing functionality

Key Enhancement ⭐

This version introduces Machine Learning integration:

- **Student Performance Prediction based on attendance**
- **Stress Level Prediction based on workload (tasks count)**

Implemented using Decision Tree Classifier

System Design

Database Models

The system consists of four main entities:

- **LectureTask:** Stores lecture schedules
- **Student:** Stores student details
- **Attendance:** Tracks attendance records linked to students
- **Wellness:** Stores teacher mood and stress logs

Relationships:

- One student can have multiple attendance records

Features of the System

🗓️ **Lecture Scheduling**

- Add subject, topic, batch, deadline, and priority
- View all scheduled lectures
- Delete completed tasks

👩‍🏫 **Attendance Management**

- Add students batch-wise
- Mark attendance using checkboxes
- Store attendance by date

📊 **Attendance Analytics**

- Calculate attendance percentage
- Display results with visual progress bars

Uses ML model to classify performance:

- Good
- Average
- Poor

🌱 **Wellness Tracking**

- Record mood and stress level
- Maintain a daily journal
- View past wellness logs

Implementation Details

- Flask routes handle different functionalities such as scheduling, attendance, and wellness
- SQLAlchemy is used to manage database operations
- Bootstrap components are used for UI styling
- Forms are used for user input and data submission

Working Flow

1. Teacher adds lecture tasks
2. System tracks number of tasks
3. AI predicts stress level
4. Teacher marks attendance
5. System calculates percentage
6. AI predicts student performance
7. Teacher logs wellness data

Challenges Faced

- Designing relationships between database tables
- Managing attendance updates efficiently
- Handling date-based records
- Ensuring clean and responsive UI
- Debugging form submissions and routing

Results and Output

The system successfully:

- Stores and manages lecture schedules
- Tracks student attendance accurately
- Generates attendance reports with percentages
- Maintains wellness logs for teachers

The application provides a unified dashboard for all functionalities.

Future Improvements

The project can be further enhanced by:

- Adding a secure login and authentication system
- Implementing role-based access (teacher/admin)
- Introducing graphical analytics (charts for attendance trends)

- Enabling data export (CSV/PDF reports)
- Integrating AI-based insights (e.g., identifying low-performing students)
- Deploying the application online for real-world usage

Conclusion

Routine Pro successfully addresses the problem of fragmented teacher management tools by providing a unified and efficient system. The project demonstrates practical application of web development concepts including backend logic, database management, and user interface design.

It also highlights the importance of integrating multiple functionalities into a single platform for improved productivity.

Learning Outcomes

Through this project, the following concepts were learned:

- Flask web development
- Database design using SQLAlchemy
- Frontend development with HTML/CSS/Bootstrap
- CRUD operations and routing
- Problem-solving and debugging
- Real-world application development

Conclusion

Routine Pro demonstrates how traditional teacher management systems can be enhanced using **web development + machine learning**. It provides a unified, efficient, and intelligent platform for managing academic workflows.